ECE

Major: Cybersecurity International

Course
Sécurité des Containers

Lab Report
Session 1

Instructor
Etienne LOUTSCH

Group Members

- ARDILLON Bastian
- SAVEAUX Octave
- BRULEY Martin

Date
February 4, 2026

Introduction à la sécurité des containers

Activités Pratiques

Q1 Lancer un Container Simple

Nous avons lancé un conteneur de test qui d'appelle hello-world avec la commande suivante :

```
docker run --rm hello-world
```

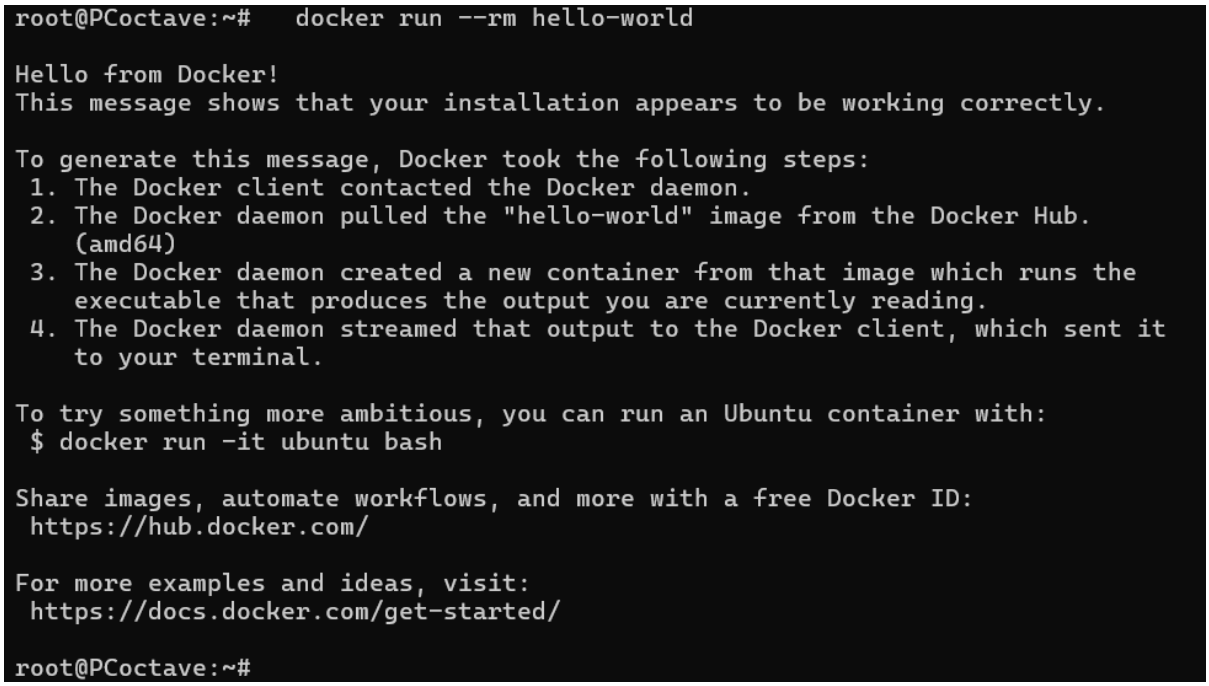

Figure 1: conteneur test hello-world

Cette commande a télécharger l'image `hello-world`, a crée un conteneur, et a executé le programme qui affiche un message de bienvenue, puis supprime le conteneur grâce à l'option `--rm`. Ce test permet de vérifier que Docker est correctement installé et opérationnel.

Q2 Explorer un Container en Interactif

Nous avons lancé un conteneur interactif basé sur l'image Alpine avec la commande suivante:

```
docker run -it --rm alpine sh
```

```
root@PCoctave:~# docker run -it --rm alpine sh
/ # pwd
/
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # cd home
/home # ls
/home # cd ..
/ # cd dev
/dev # ls
console  fd       mqueue   ptmx     random   stderr   stdout   urandom
core     full     null     pts      shm      stdin    tty      zero
/dev # whoamI
sh: whoamI: not found
/dev # whoami
root
/dev #
```

Figure 2: exploration du conteneur Alpine en mode interactif

Les options `-it` permettent d'ouvrir un shell interactif à l'intérieur du conteneur. Nous avons ensuite testé des commandes Linux classiques (dont `ls`, `pwd`, `whoami`, `cd`).

Q3 Analyser les ressources système d'un container

Nous avons lancé un conteneur nginx en arrière-plan puis nous avons affiché ses ressources avec les commandes suivantes:

```
docker run -d --name test-container nginx
docker stats test-container
```

```
root@PCoctave:~# docker run -d --name test-container nginx
58fafdb22becdf7816ce4734d3aa3d49812ae4521514ee99f39a456d1c1b93ca
root@PCoctave:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS     NAMES
58fafdb22bec   nginx     "/docker-entrypoint.…"   8 seconds ago   Up 8 seconds   80/tcp    test-container
root@PCoctave:~#
```

Figure 3: lancement du conteneur nginx en arrière-plan

```
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
f8b765713d97   test-container   0.00%     7.629MiB / 7.655GiB   0.10%     1.17kB / 126B   0B / 12.3kB   9
```

Figure 4: affichage des ressources du conteneur nginx

L'option `-d` exécute le conteneur en arrière-plan, l'option `--name` permet de lui donner un nom. La commande `docker stats` affiche en temps réel la consommation CPU, la mémoire utilisée, l'utilisation réseau, le I/O et le PID du conteneur. On constate que le conteneur nginx consomme 7.6 MB de RAM et 0.00% de CPU. Ce conteneur est donc assez léger.

On constate que même un conteneur léger comme Nginx consomme de la RAM et un peu de CPU. Le suivi des ressources permet de détecter des fuites mémoire ou une surcharge et d'ajuster les limites (mémoire, CPU) si nécessaire pour la sécurité et la stabilité de l'hôte.

Q4 Lister les capacités d'un container

Pour vérifier les capacités du processus dans le conteneur nous avons utilisé la commande suivante:

```
docker run --rm --cap-add=SYS_ADMIN alpine sh -c 'cat /proc/self/status | grep Cap'
```

```
root@PCoctave:~# docker run --rm --cap-add=SYS_ADMIN alpine sh -c 'cat /proc/self/status'
Name:   cat
Umask:  0022
State:  R (running)
Tgid:   1
Ngid:   0
Pid:    1
PPid:   0
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups: 0 1 2 3 4 6 10 11 20 26 27
NStgid: 1
NSpid:  1
NSpgid: 1
NSsid:  1
Kthread:        0
VmPeak:     1688 kB
VmSize:     1688 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:       768 kB
VmRSS:       768 kB
RssAnon:               0 kB
RssFile:             768 kB
RssShmem:              0 kB
VmData:       88 kB
VmStk:       132 kB
VmExe:       612 kB
VmLib:       368 kB
VmPTE:        44 kB
VmSwap:        0 kB
HugetlbPages:          0 kB
CoreDumping:    0
THP_enabled:    1
untag_mask:     0xffffffffffffffff
Threads:        1
SigQ:   1/31313
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000004
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 00000000a82425fb
CapEff: 00000000a82425fb
CapBnd: 00000000a82425fb
CapAmb: 0000000000000000
NoNewPrivs:     0
Seccomp:        2
Seccomp_filters:        2
Speculation_Store_Bypass:       thread vulnerable
SpeculationIndirectBranch:      conditional enabled
Cpus_allowed:   ff
Cpus_allowed_list:      0-7
Mems_allowed:   00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000001
Mems_allowed_list:      0
voluntary_ctxt_switches:        43
nonvoluntary_ctxt_switches:     1
root@PCoctave:~#
```

Figure 4: capacités Linux visibles dans /proc/self/status

il faudrait mettre des infos en plus mais vasy jsp quoi mettre a part avec ai

Vulnérabilités et Menaces

Activités Pratiques

Q1 Tester un Container avec des Permissions Élevées

Nous avons lancé un conteneur en mode privilégié avec la commande suivante:

```
docker run --rm --privileged alpine sh -c 'echo hello from privileged mode'
```

```
root@PCoctave:~# docker run --rm --privileged alpine sh -c 'echo hello from privileged mode'
hello from privileged mode
root@PCoctave:~#
root@PCoctave:~#
root@PCoctave:~#
```

Figure 5: conteneur en mode privilégié

```
root@PCoctave:~# docker run -it --rm --privileged alpine sh
/ # ls /dev
autofs          loop3           ram4            stdout          tty31           tty56           vcs4
bsg             loop4           ram5            tty             tty32           tty57           vcs5
btrfs-control   loop5           ram6            tty0            tty33           tty58           vcs6
console         loop6           ram7            tty1            tty34           tty59           vcsa
core            loop7           ram8            tty10           tty35           tty6            vcsa1
cpu_dma_latency mapper          ram9            tty11           tty36           tty60           vcsa2
cuse            mcelog          random          tty12           tty37           tty61           vcsa3
dxg             mem             rtc0            tty13           tty38           tty62           vcsa4
fd              mptctl          sda             tty14           tty39           tty63           vcsa5
full            mqueue          sdb             tty15           tty4            tty7            vcsa6
fuse            net             sdc             tty16           tty40           tty8            vcsu
hpet            null            sdd             tty17           tty41           tty9            vcsu1
hvc0            nvram           sdg             tty18           tty42           ttyS0           vcsu2
hvc1            ppp             sdh             tty19           tty43           ttyS1           vcsu3
hvc2            ptmx            sdi             tty2            tty44           ttyS2           vcsu4
hvc3            ptp0            sg0             tty20           tty45           ttyS3           vcsu5
hvc4            pts             sg1             tty21           tty46           ttyS4           vcsu6
hvc5            ram0            sg2             tty22           tty47           ttyS5           vfio
hvc6            ram1            sg3             tty23           tty48           ttyS6           vga_arbiter
hvc7            ram10           sg4             tty24           tty49           ttyS7           vhost-net
hwrng           ram11           sg5             tty25           tty5            uinput          vhost-vsock
kmsg            ram12           sg6             tty26           tty50           urandom         vport0p0
kvm             ram13           shm             tty27           tty51           userfaultfd     vport0p1
loop-control    ram14           snapshot        tty28           tty52           vcs             vport0p2
loop0           ram15           snd             tty29           tty53           vcs1            vsock
loop1           ram2            stderr          tty3            tty54           vcs2            zero
loop2           ram3            stdin           tty30           tty55           vcs3
/ #
```

Figure 6: "ls /dev" dans un conteneur privilégié

Pour voir pourquoi cette pratique est dangereuse, nous avons comparé avec un conteneur classique. Dans un conteneur lancé en privilégié, la commande `ls /dev` affiche une liste très longue de devices de l'hôte (disques `sda`, `sdb`, devices loop, RAM, `kvm`), le conteneur a donc accès au matériel et aux interfaces noyau de la machine hôte, ce qui est tres dangereux.

```
/ # cat /proc/self/status | grep Cap
CapInh: 0000000000000000
CapPrm: 000001ffffffffff
CapEff: 000001ffffffffff
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000
/ #
```

Figure 7: capacités CapPrm, CapEff, CapBnd en mode privilégié

De même, `cat /proc/self/status | grep Cap` montre que les capacités CapPrm, CapEff et CapBnd sont quasi au maximum (valeur hexadécimale `000001ffffffffff`), le processus dispose donc de presque toutes les capacités Linux. Un attaquant qui compromet ce conteneur peut ainsi accéder aux disques, à la mémoire ou au noyau de l'hôte.

Q2 Simuler une Évasion de Container

Nous avons lancé un conteneur et monté le système de fichiers de l'hôte dans le conteneur avec la commande suivante:

```
docker run --rm -v /:/mnt alpine sh -c 'ls /mnt'
```

```
root@PCoctave:~# docker run --rm -v /:/mnt alpine sh -c 'ls /mnt'
Docker
bin
boot
dev
etc
home
init
lib
lib32
lib64
libx32
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
```

Figure 8: liste du système de fichiers de l'hôte via le montage /:/mnt

Le conteneur voit ainsi l'intégralité du système de fichiers de la machine hôte (répertoires système, home, données sensibles, ...). Un attaquant ayant accès au conteneur peut lire et écrire les fichiers de l'hôte, installer des backdoors, voler des secrets ou des clés. Pour limiter le risque, on peut monter en lecture seule avec `:ro`, ce qui permet au conteneur de voir le système de fichiers de l'hôte mais empêche toute écriture.

```
root@PCoctave:~# docker run -it --rm -v /:/mnt:ro alpine sh
/ #
/ #
/ # ls mnt
Docker      dev         init        lib64       media       proc        sbin        sys         var
bin         etc         lib         libx32      mnt         root        snap        tmp
boot        home        lib32       lost+found  opt         run         srv         usr
/ #
/ # echo "test" >> mnt/etc/host
host.conf  hostname   hosts
/ # echo "test" >> mnt/etc/hostname
sh: can't create mnt/etc/hostname: Read-only file system
/ #
```

Figure 9: liste du système de fichiers de l'hôte via le montage /:/mnt en lecture seule

Q3 Créer une Image Sécurisée

Nous avons créé un Dockerfile minimaliste avec `nano Dockerfile`, puis construit et exécuté l'image comme suit.

```
FROM alpine
RUN adduser -D appuser
```

```
USER appuser
CMD ["echo", "Container sécurisé!"]
```

Construction de l'image et affichage de l'id et de l'uid de l'utilisateur:

```
docker build -t secure-alpine
docker run --rm secure-alpine sh -c 'echo Container Secu && id'
```

```
root@PCoctave:~#
root@PCoctave:~# nano Dockerfile
root@PCoctave:~# docker build -t secure-alpine .
[+] Building 0.1s (6/6) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 123B                                                                          0.0s
 => [internal] load metadata for docker.io/library/alpine:latest                                              0.0s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c  0.0s
 => => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c  0.0s
 => CACHED [2/2] RUN adduser -D appuser                                                                       0.0s
 => exporting to image                                                                                        0.0s
 => => exporting layers                                                                                       0.0s
 => => exporting manifest sha256:ec3c475d0b4b50ad61b6df433b74ccc37b269ea896a9bca1947f81edfa710b46             0.0s
 => => exporting config sha256:da1a528a2778e6b3dd8702f5c4e823daefd7f18bae492f75b8077315d396f553               0.0s
 => => exporting attestation manifest sha256:1dd8971f9fa0d2c02f3fd4262926d3c3f7396cb4272c2f6a1e5a982e2ab68db4  0.0s
 => => exporting manifest list sha256:39028e82e2d10fb386500bcd42e9ceda6dc3aa5358c87a2f224ddf91f773bbe0        0.0s
 => => naming to docker.io/library/secure-alpine:latest                                                       0.0s
 => => unpacking to docker.io/library/secure-alpine:latest                                                    0.0s
root@PCoctave:~#
root@PCoctave:~# docker images
REPOSITORY                    TAG       IMAGE ID       CREATED         SIZE
secure-alpine                 latest    39028e82e2d1   4 minutes ago   13MB
nginx                         latest    9dd288848f44   37 hours ago    237MB
alpine                        latest    25109184c71b   7 days ago      13MB
gcr.io/k8s-minikube/kicbase   v0.0.48   41454ef774d0   4 months ago    1.84GB
gcr.io/k8s-minikube/kicbase   <none>    7171c97a5162   4 months ago    1.84GB
hello-world                   latest    05813aedc15f   5 months ago    20.3kB
root@PCoctave:~# docker run --rm secure-alpine sh -c 'echo Container Secu && id'
Container Secu
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
root@PCoctave:~# docker run --rm secure-alpine
Container sécurisé!
root@PCoctave:~#
```

Figure 9: construction de secure-alpine, affichage de l'id/uid de appuser et exécution du CMD

Quand on lance le conteneur, il affiche que l'utilisateur actif s'appelle "appuser" avec l'identifiant 1000. Ensuite, le message "Container sécurisé!" apparaît. Utiliser un utilisateur qui n'est pas root rend le conteneur plus sûr, car même si quelqu'un arrive à l'attaquer, il n'aura pas tous les pouvoirs de droits root dans le conteneur.

Restreindre l'accès réseau d'un container

Q5 Bloquer la connexion internet dans un container

Q6 Tester l'accès internet avec par exemple ping google.com

Pour bloquer la connexion Internet d'un conteneur, on utilise la déconnexion du réseau bridge :

```
docker network disconnect bridge mon-container
```

```
arbasti@arbasti-Desktop:~$ docker run -it --rm --name disconnected-container secure-alpine sh
/ $ ping -c 3 google.com
PING google.com (142.250.179.110): 56 data bytes
64 bytes from 142.250.179.110: seq=0 ttl=42 time=8.386 ms
64 bytes from 142.250.179.110: seq=1 ttl=42 time=31.695 ms
64 bytes from 142.250.179.110: seq=2 ttl=42 time=20.579 ms

--- google.com ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 8.386/20.220/31.695 ms
/ $
/ $ ping -c 3 google.com
ping: bad address 'google.com'
/ $
```

```
arbasti@arbasti-Desktop: ~
arbasti@arbasti-Desktop:~$ docker network disconnect bridge disconnected-container
arbasti@arbasti-Desktop:~$
```

Figure 11: blocage de l'accès Internet par déconnexion du bridge

Le conteneur n'a plus d'interface connectée au réseau bridge et ne peut plus atteindre l'extérieur. La connexion Internet est ainsi bloquée.

Q7 Télécharger et Scanner une Image

Nous avons téléchargé une image volontairement vulnérable puis l'avons analysée avec Trivy :

```
docker pull vulnerables/web-dvwa
trivy image vulnerables/web-dvwa -f json -o trivy_report.json
```

L'image `vulnerables/web-dvwa` est une application web d'entraînement aux failles (DVWA). Trivy scanne les couches de l'image (OS et dépendances) et détecte les CVE connues. L'option `-f json -o trivy_report.json` enregistre le rapport au format JSON pour exploitation ultérieure.

Pour obtenir un résumé par niveau de sévérité à partir du rapport JSON :

```
cat trivy_report.json | jq '.Results[].Vulnerabilities[] | .Severity' | sort |
uniq -c
```

Résumé des vulnérabilités de cette image : le comptage par sévérité donne notamment : 254 CRITICAL, 551 HIGH, 642 MEDIUM, 116 LOW et 12 UNKNOWN. L'image présente donc un nombre très élevé de failles connues. Une image de ce type ne doit pas être utilisée en production ; elle sert à tester les outils de scan (Trivy, Grype) et à prendre conscience de l'importance d'analyser les images avant déploiement.

```
root@PCoctave:~# cat trivy_report.json | jq '.Results[].Vulnerabilities[] | .Severity' | sort | uniq -c
jq: error (at <stdin>:147805): Cannot iterate over null (null)
    254 "CRITICAL"
    551 "HIGH"
    116 "LOW"
    642 "MEDIUM"
     12 "UNKNOWN"
root@PCoctave:~#
```

Figure 12: comptage des vulnérabilités par sévérité à partir de trivy_report.json

Q8 Scanner une Image pour Détecter les Vulnérabilités

Nous avons analysé une image avec Grype (un scanneur de vulnérabilités) :

```
grype alpine:latest
```

```
root@PCoctave:~# grype alpine:latest
 ✔Vulnerability DB                [updated]
 ✔Loaded image                                                                                    alpine:latest
 ✔Parsed image                                sha256:a40c03cbb81c59bfb0e0887ab0b1859727075da7b9cc576a1cec2c771f38c5fb
 ✔Cataloged contents                                 ceedcaa851833940d364da9a0a99b8c50699bef7776439e2fb3d923a60a6638b
   ├── ✔Packages                        [16 packages]
   ├── ✔Executables                     [17 executables]
   ├── ✔File metadata                   [79 locations]
   └── ✔File digests                    [79 files]
 ✔Scanned for vulnerabilities     [3 vulnerability matches]
   ├── by severity: 0 critical, 0 high, 3 medium, 0 low, 0 negligible
NAME           INSTALLED   TYPE  VULNERABILITY   SEVERITY  EPSS           RISK
busybox        1.37.0-r30  apk   CVE-2025-60876  Medium    < 0.1% (16th)  < 0.1
busybox-binsh  1.37.0-r30  apk   CVE-2025-60876  Medium    < 0.1% (16th)  < 0.1
ssl_client     1.37.0-r30  apk   CVE-2025-60876  Medium    < 0.1% (16th)  < 0.1
root@PCoctave:~#
```

Figure 13: analyse de l'image avec Grype

Grype affiche les CVE détectées dans les packages de l'image.

Différences entre Grype et Trivy :

- **Bases de données :** Grype s'appuie sur la base Anchore alors que Trivy utilise sa propre base. Les listes de CVE et les niveaux de sévérité peuvent légèrement différer.
- **Formats de sortie :** Trivy propose de nombreux formats (table, JSON, SARIF, etc.) et une option `-f json -o fichier` pratique pour automatisation. Grype offre aussi du JSON et une sortie en tableau.
- **Usage :** Les deux permettent de scanner des images Docker, des fichiers, des répertoires. Trivy est souvent utilisé pour l'intégration CI/CD et Grype fait partie de l'écosystème Anchore/Syft.
- **Couverture :** Grype est plus rapide et a une couverture plus large, alors que Trivy est plus complet et a une couverture plus restreinte. Il est utile de croiser les deux outils pour une vue plus complète.

Q9 Dockerfile - Run as User

Nous avons créé un Dockerfile avec un utilisateur dédié pour une application Node.js dans le répertoire `~/node-app` :

```
FROM node:18-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY . .
RUN chown -R appuser:appgroup /app
USER appuser
CMD ["node", "app.js"]
```

Le conteneur affiche la sortie de l'application. Pour vérifier que le processus ne tourne pas en root, nous sommes entrés dans le conteneur et avons lancé `whoami && id`. La sortie est `appuser` puis `uid=100(appuser) gid=101(appgroup) groups=101(appgroup)` : l'UID est 100 et le GID 101, ce qui confirme l'exécution sous l'utilisateur dédié.

```
/app $ whoami && id
appuser
uid=100(appuser) gid=101(appgroup) groups=101(appgroup)
/app $
```

Figure 14: vérification de l'utilisateur (whoami && id) — appuser, uid=100, gid=101

Après construction de l'image (`docker build -t node-app-secure .`), nous avons exécuté le conteneur :

```
docker run --rm node-app-secure
```

Le screenshot ci-dessous montre l'exécution de cette commande depuis l'hôte : le conteneur démarre correctement et affiche la sortie de l'application : `test`.

```
root@PCoctave:~/node-app#
root@PCoctave:~/node-app#
root@PCoctave:~/node-app# docker run --rm node-app-secure
test
root@PCoctave:~/node-app#
```

Figure 15: exécution de `docker run --rm node-app-secure` (sortie : test)

Q10 Dockerfile - Run light!

Nous avons créé deux Dockerfiles pour une application en Golang qui affiche du texte.

Premier Dockerfile :

```
FROM golang:1.19 AS builder
WORKDIR /app
COPY main.go .
RUN go build -o monapp main.go
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/monapp /
CMD ["/monapp"]
```

Le build se fait dans une image Go, puis seul le binaire est copié dans une image Distroless.

Deuxième Dockerfile :

```
FROM golang:1.19
WORKDIR /app
COPY main.go .
RUN go build -o monapp main.go
CMD ["./monapp"]
```

Toute l'image reste basée sur Golang, ce qui donne une image beaucoup plus lourde.

Comparaison des tailles : Après construction des deux images, la commande `docker images` exécutée depuis `~/go-app` donne notamment : **go-app-full ≈ 1,53 Go** et **go-app-light ≈ 9,15 Mo**. La version multi-stage avec Distroless réduit fortement la taille et la surface d'attaque. Les deux conteneurs ont été exécutés et affichent correctement le message de l'application.